

CONTROLS SYSTEMS

Project 03

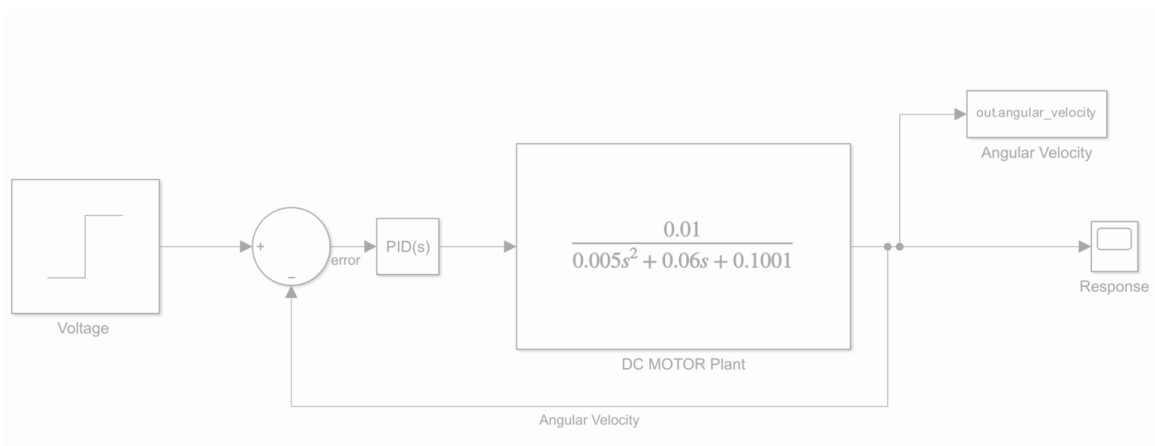
PID Speed Control of a DC Motor Using MATLAB and Simulink

Designed, Developed and Documented by

Zohaib Imtiaz

MATLAB R2024b • Control System Toolbox • Aerospace Engineering

July 2026



Copyright © 2026 Zohaib Imtiaz. All Rights Reserved.

This report is the original work of the author and has been prepared for academic purposes. No part of this document may be reproduced, distributed, or transmitted in any form without proper acknowledgment of the author, except where permitted for educational or research purposes.

Abstract

This project presents the design, implementation, and analysis of a PID (Proportional-Integral-Derivative) speed controller for an armature-controlled DC motor using MATLAB and Simulink. Building directly on the open-loop motor model developed in Project 02, this project introduces the concept of closed-loop feedback control and systematically investigates how each control action — proportional, integral, and derivative — influences the dynamic performance of the motor. The open-loop system was first verified in Simulink, confirming a steady-state speed of 0.0998 rad/s and a steady-state error of 90.02% for a 1 V step input. A unity feedback closed-loop system was then constructed, followed by individual P, PI, and PD controller studies, each involving multiple gain values. The interactions between gains were analyzed to understand integrator windup, breakaway behavior, and derivative over-braking. A final PID controller with gains $K_p=10$, $K_i=20$, $K_d=0.1$ was manually tuned to meet all performance targets: rise time under 1 second, settling time under 3 seconds, overshoot below 5%, and zero steady-state error. This was compared against MATLAB's automatic `pdtune()` result ($K_p=19.5$, $K_i=53$, $K_d=1.57$), revealing a fundamental trade-off between speed and smoothness. Disturbance rejection tests confirmed that the tuned PID controller maintains exactly 1.0 rad/s despite sudden load inputs of ± 0.5 applied at $t=5$ s. The project establishes a complete understanding of PID control theory and directly motivates the next step: applying these principles to real aerospace plants beginning with aircraft pitch control in Project 04.

Table of Contents

Abstract	iii
List of Tables	vii
Table of Figures	viii
1 Objectives	1
2 Introduction.....	2
3 System Description	3
3.1 Nominal Parameters.....	3
3.2 System Characteristics	4
4 Methodology	4
5 Results and Discussion	5
5.1 Open Loop Verification in Simulink	5
5.1.1 Simulink Block Diagram	5
5.1.2 Open Loop Step Response	5
5.1.3 Performance Specifications	6
5.1.4 Observation	6
5.2 Closed Loop System with Unity Feedback.....	7
5.2.1 Simulink Block Diagram	7
5.2.2 Closed Loop Unity Feedback Step Response	7
5.2.3 Performance Specifications	8
5.2.4 Observation	8
5.3 P Controller Analysis.....	9
5.3.1 P Controller Simulink Block Diagram.....	9
5.3.2 P Controller Step Responses $K_p = 1$	9
5.3.3 P Controller Simulink Block Diagram for varying K_p	10
5.3.4 P Controller Step Responses	10

5.3.5	Performance Specifications	11
5.3.6	Observations:	11
5.4	PI Controller Analysis.....	12
5.4.1	PI Controller Simulink Block Diagram	12
5.4.2	PI Controller Simulink Block Diagram for varying K_i	12
5.4.3	PI Controller Step Responses for All K_i Values.....	13
5.4.4	Performance Specifications	13
5.4.5	Observations	14
5.5	PD Controller Analysis	15
5.5.1	PD Controller Simulink Block Diagram.....	15
5.5.2	PD Controller Simulink Block Diagram for varying K_d	15
5.5.3	PD Controller Step Responses for All K_d Values	16
5.5.4	Performance Specifications	16
5.5.5	Observations:	17
5.6	Full PID Controller Tuning.....	18
5.6.1	PID Controller Simulink Block Diagram	18
5.6.2	PID Step Responses — Three Gain Combinations.....	19
5.6.3	Performance Specifications	19
5.6.3.1	Comparison of different combinations	19
5.6.3.2	Selected Gains.....	20
5.7	Complete Controller Comparison.....	20
5.7.1	Observations	21
5.8	pdtune() Automatic Tuning Comparison.....	21
5.8.1	Performance Specifications	21
5.8.2	Gain Comparison Analysis:	22
5.9	Disturbance Rejection.....	23

5.9.1	Disturbance Rejection Simulink Block Diagram.....	23
5.9.2	Disturbance Rejection Response.....	24
5.9.3	Disturbance Rejection Results	24
5.9.4	Observations:	25
6	Aerospace Relation	26
7	Conclusions.....	27
8	Future Work	28
9	Appendix.....	31
9.1	MATLAB Code	31

List of Tables

<i>Table 1 Nominal Plant Parameters</i>	<i>3</i>
<i>Table 2 Open Loop Performance Specifications — Simulink Verification.....</i>	<i>6</i>
<i>Table 3 Comparison — Open Loop vs Unity Feedback Closed Loop</i>	<i>8</i>
<i>Table 4 P Controller Performance Specifications for Varying K_p</i>	<i>11</i>
<i>Table 5 PI Controller Performance Specifications — Varying K_i with $K_p=10$.....</i>	<i>13</i>
<i>Table 6 PD Controller Performance Specifications — Varying K_d with $K_p=10$</i>	<i>16</i>
<i>Table 7 PID Tuning Comparison — Three Gain Combinations</i>	<i>19</i>
<i>Table 8 Final PID Tuning — Performance Target Verification</i>	<i>20</i>
<i>Table 9 Complete Controller Comparison — All Configurations.....</i>	<i>20</i>
<i>Table 10 Manual PID Tuning vs pidtune() Automatic Tuning</i>	<i>21</i>
<i>Table 11 Disturbance Rejection Results — Final Speed After Recovery</i>	<i>24</i>

Table of Figures

Figure 1 Simulink Block Diagram — Open Loop DC Motor	5
Figure 2 Open Loop Step Response in Simulink.....	5
Figure 3 Simulink Block Diagram — Closed Loop Unity Feedback (No Controller)	7
Figure 4 Step Response — Closed Loop with Unity Feedback.....	7
Figure 5 Simulink Block Diagram — P Controller	9
Figure 6 Step Responses for P Controller — $K_p = 0.5, 1, 5, 10, 50$	9
Figure 7 Simulink Block Diagram P Controller — $K_p = 0.5, 1, 5, 10, 50$	10
Figure 8 Step Responses for P Controller — $K_p = 0.5, 1, 5, 10, 50$	10
Figure 9 Simulink Block Diagram — PI Controller	12
Figure 10 Simulink Block Diagram — PI Controller — $K_i = 1, 5, 10, 20, 30, 50$ ($K_p=10$)	12
Figure 11 Step Responses for PI Controller — $K_i = 1, 5, 10, 20, 30, 50$ ($K_p=10$)	13
Figure 12 Simulink Block Diagram — PD Controller	15
Figure 13 Simulink Block Diagram — PD Controller — $K_d = 0.01, 0.05, 0.1, 0.5, 5, 10$ ($K_p=10$).....	15
Figure 14 Step Responses for PD Controller — $K_d = 0.01, 0.05, 0.1, 0.5, 5, 10$ ($K_p=10$)	16
Figure 15 Simulink Block Diagram — Full PID Controller.....	18
Figure 16 Step Responses for Three PID Gain Combinations	19
Figure 17 Simulink Block Diagram — Disturbance Rejection Test	23
Figure 18 Motor Speed Response with Disturbance at $t=5s$	24

1 Objectives

The specific objectives are:

- Understand why open-loop control is insufficient and establish the need for closed-loop feedback control.
- Build and verify the open-loop DC motor model in Simulink and confirm it matches the MATLAB transfer function results from Project 02.
- Construct a unity feedback closed-loop system and observe the effect of feedback without a controller.
- Design and analyze a Proportional (P) controller by varying K_p across multiple values and recording the effect on rise time, settling time, overshoot, and steady-state error.
- Design and analyze a Proportional-Integral (PI) controller by varying K_i while holding K_p constant, and understand the role of integral action in eliminating steady-state error.
- Design and analyze a Proportional-Derivative (PD) controller by varying K_d while holding K_p constant, and understand the role of derivative action in reducing overshoot.
- Tune a complete PID controller to meet four specific performance targets simultaneously.
- Compare manual PID tuning against MATLAB's automatic `pidentune()` and discuss the engineering trade-offs.
- Perform disturbance rejection tests to verify that the closed-loop system can recover from sudden load inputs.
- Connect the PID controller design experience to real aerospace actuator control systems.

2 Introduction

In Project 02, the DC motor was modeled as an open-loop plant. The analysis revealed that for a 1 V step input, the motor reached only 0.0998 rad/s — a steady-state error of approximately 90%. The motor parameters, not the desired speed, determined the final output. This is the fundamental limitation of open-loop control: **the system has no mechanism to detect or correct the difference between what was commanded and what was achieved.**

Project 03 addresses this limitation by introducing feedback control. A sensor measures the actual motor speed and feeds it back to a comparator, which calculates the error between the reference command and the measured output. A PID controller then uses this error signal to generate a corrective input to the motor. As long as an error exists, the controller continues to apply corrections — and with integral action, it accumulates those corrections until the error is driven to exactly zero.

The PID controller is the most widely used controller in industry and aerospace. Despite its simplicity, it is found in aircraft autopilots, satellite attitude control systems, rocket thrust vector controllers, UAV flight control systems, and industrial servo drives. Understanding how to tune PID gains, predict their effects, and interpret the resulting behavior is a core skill for any control engineer.

This project follows a systematic approach: each control action is studied in isolation before combining them. This methodology makes the contribution of each term physically clear and prevents the common beginner mistake of tuning all three gains simultaneously without understanding what each one is doing.

3 System Description

The plant used in this project is the same armature-controlled DC motor from Project 02. The transfer function relating the input voltage $V(s)$ to the output angular velocity $\omega(s)$ is given by:

$$G(s) = \frac{\omega(s)}{V(s)} = \frac{K_t}{JLs^2 + (JR + bL)s + (bR + K_tK_e)}$$

3.1 Nominal Parameters

The following nominal parameter values were used throughout the base-case analysis:

Table 1 Nominal Plant Parameters

Parameter	Symbol	Value	Unit
Moment of Inertia	J	0.01	kg·m ²
Viscous Friction	b	0.1	N·m·s/rad
Armature Resistance	R	1	Ω
Armature Inductance	L	0.5	H
Torque Constant	K _t	0.01	N·m/A
Back-EMF Constant	K _e	0.01	V·s/rad

With the nominal parameters substituted, the transfer function becomes:

$$G(s) = \frac{0.01}{0.005s^2 + 0.06s + 0.1001}$$

3.2 System Characteristics

- **Poles:** The open-loop system has two real negative poles at -9.9975 and -2.0025 .
- **Response:** This pole configuration produces an **overdamped response** with **zero overshoot**.
- **Steady-State Output:** The system yields a steady-state angular velocity of 0.0998 rad/s for a 1 V step input.

4 Methodology

All controller designs were implemented in Simulink using the following block types:

Step, Sum, Gain, Transfer Function, PID Controller, To Workspace, and Scope.

The simulation time was set to 10 seconds for single-controller tests and 15 seconds for disturbance rejection tests.

The methodology followed this sequence for every controller type:

Step 1 Predict: Before running any simulation, the expected effect on rise time, settling time, overshoot, and steady-state error was written in a notebook.

Step 2 Simulate: The Simulink model was run and the scope output was captured.

Step 3 Quantify: The `stepinfo()` command was used in MATLAB to extract exact numerical performance specifications from the workspace data.

Step 4 Explain: The result was physically interpreted and compared against the prediction.

This predict-simulate-explain loop was maintained throughout all controller studies to develop engineering intuition rather than simply recording numbers.

5 Results and Discussion

5.1 Open Loop Verification in Simulink

The first step was to verify that the Simulink model produces the same response as the MATLAB transfer function analysis from Project 02. A step input of 1 V was applied to the motor transfer function with no feedback or controller.

5.1.1 Simulink Block Diagram

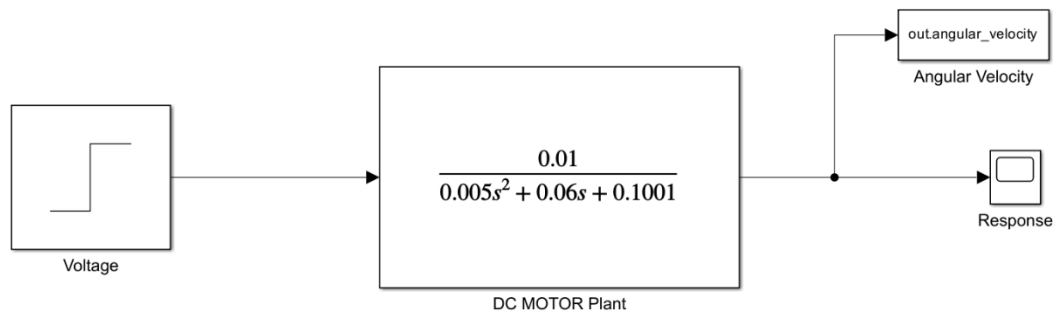


Figure 1 Simulink Block Diagram — Open Loop DC Motor

5.1.2 Open Loop Step Response

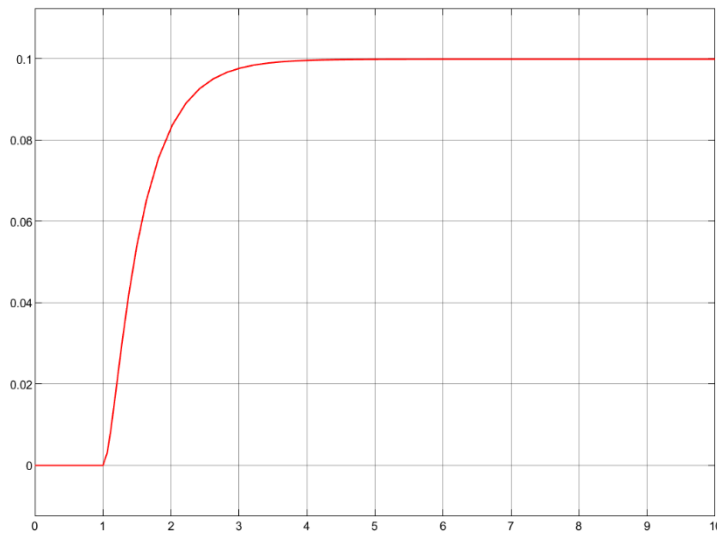


Figure 2 Open Loop Step Response in Simulink

5.1.3 Performance Specifications

Table 2 Open Loop Performance Specifications — Simulink Verification

Parameter	Value
Rise Time	1.143 s
Settling Time	3.073 s
Overshoot	0%
Steady-State Speed	0.0999 rad/s
Steady-State Error	90.01%

5.1.4 Observation

The Simulink result matches the MATLAB result from Project 02 (steady-state speed = 0.0998 rad/s) to within numerical solver tolerance. This confirms that the Simulink model is correctly implemented and can be used as the foundation for all subsequent controller designs.

The steady state error of 90.01% confirms the core motivation for this project. If this motor were driving an aircraft elevator actuator commanded to move 1 degree, it would actually move only 0.1 degrees. This is entirely unacceptable in a safety-critical aerospace system and demonstrates why feedback control is not optional — it is mandatory.

5.2 Closed Loop System with Unity Feedback

Before adding any controller, a unity feedback loop was closed around the plant. The output angular velocity was measured and subtracted from the reference to form an error signal, which was fed directly into the motor.

5.2.1 Simulink Block Diagram

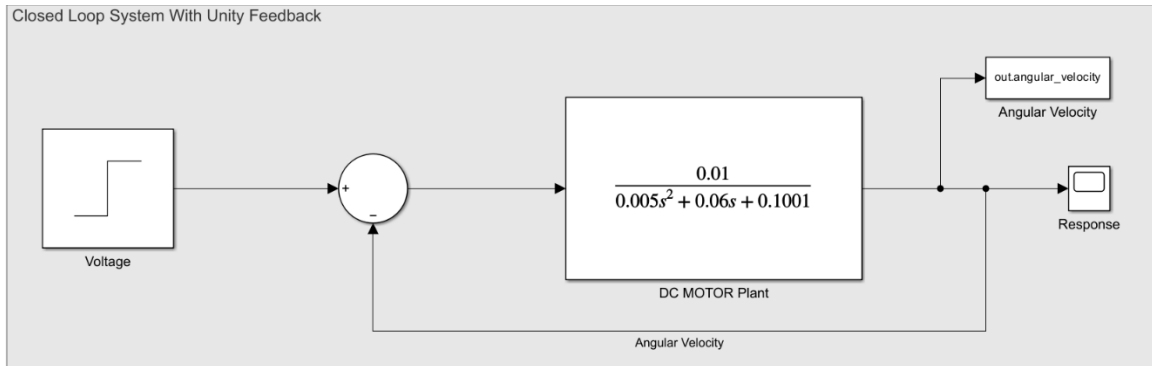


Figure 3 Simulink Block Diagram — Closed Loop Unity Feedback (No Controller)

5.2.2 Closed Loop Unity Feedback Step Response

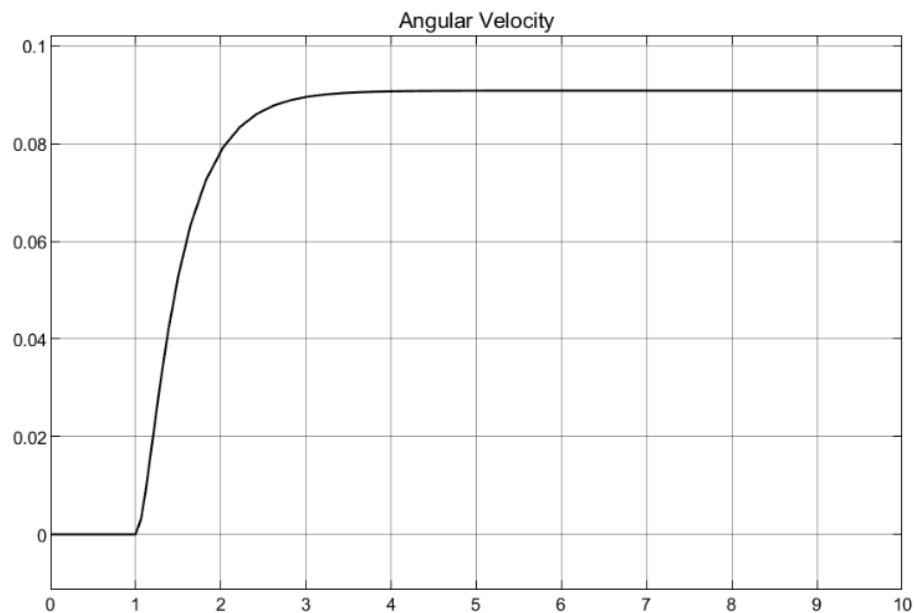


Figure 4 Step Response — Closed Loop with Unity Feedback

5.2.3 Performance Specifications

Table 3 Comparison — Open Loop vs Unity Feedback Closed Loop

Parameter	Open Loop	Closed Loop (Unity)
Rise Time	1.143 s	1.027 s
Settling Time	3.073 s	2.852 s
Overshoot	0%	0%
Steady-State Speed	0.0999	0.0908
Steady-State Error	90.01%	90.92%

5.2.4 Observation

The result is counterintuitive — closing the loop with unity feedback actually increased the steady-state error from 90.01% to 90.92%, and the final speed dropped from 0.0999 to 0.0908 rad/s. This happened because unity feedback with no controller is mathematically equivalent to a P controller with $K_p=1$. The feedback loop subtracts the output from the reference, reducing the effective input to the motor. The closed-loop steady state can be derived analytically as:

$$T(0) = \frac{G(0)}{1 + G(0)} = \frac{0.0998}{1 + 0.0998} = 0.09074 \approx 0.0908$$

which matches exactly.

“This result proves a critical point: feedback alone does not fix steady-state error. A controller is required to provide the corrective action needed to drive the error to zero.”

5.3 P Controller Analysis

A Gain block with value K_p was inserted between the error signal and the plant. Five values were tested: $K_p = 0.5, 1, 5, 10, 50$.

The closed-loop transfer function with P control is:

$$T(s) = \frac{K_p G(s)}{1 + K_p G(s)}$$

The steady-state value is:

$$\omega_{ss} = T(0) = \frac{K_p G(0)}{1 + K_p G(0)}$$

5.3.1 P Controller Simulink Block Diagram

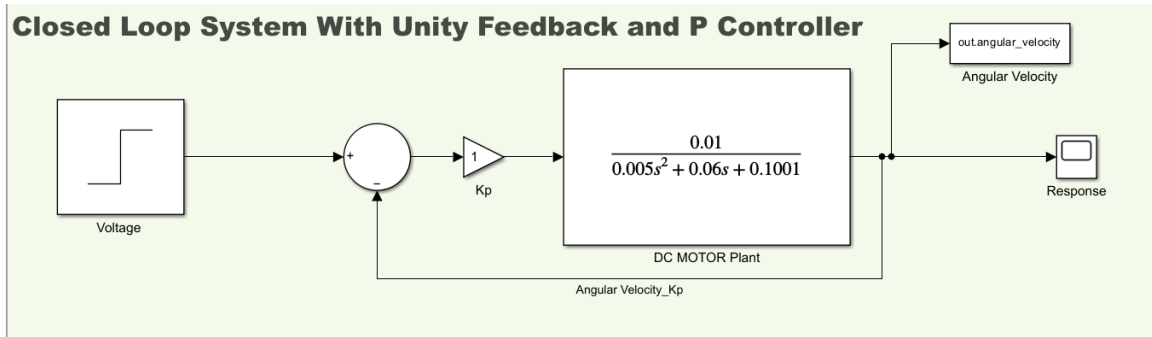
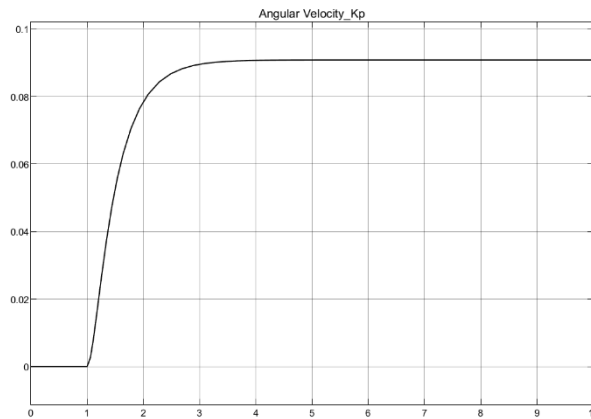


Figure 5 Simulink Block Diagram — P Controller

5.3.2 P Controller Step Responses $K_p = 1$



Note: $K_p = 1$ gives the same behavior of a system without any controller

Figure 6 Step Responses for P Controller — $K_p = 0.5, 1, 5, 10, 50$

5.3.3 P Controller Simulink Block Diagram for varying Kp

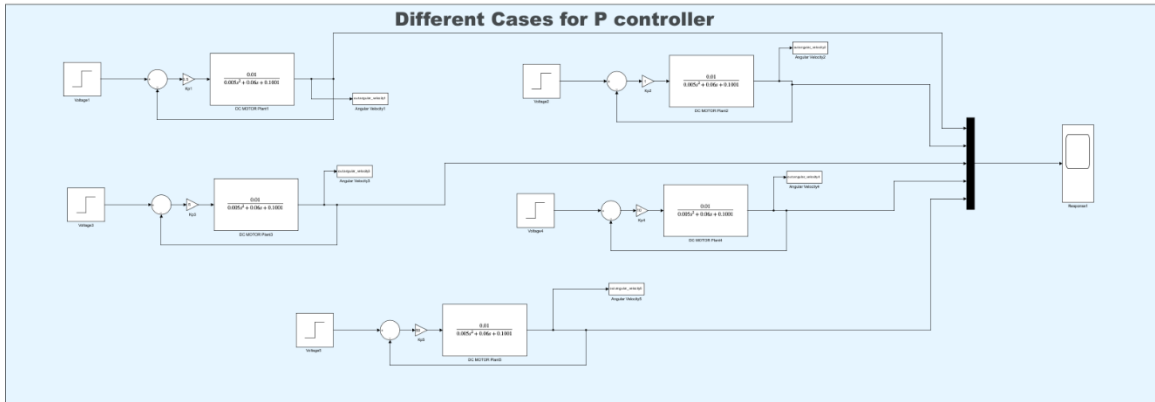


Figure 7 Simulink Block Diagram P Controller — $K_p = 0.5, 1, 5, 10, 50$

5.3.4 P Controller Step Responses

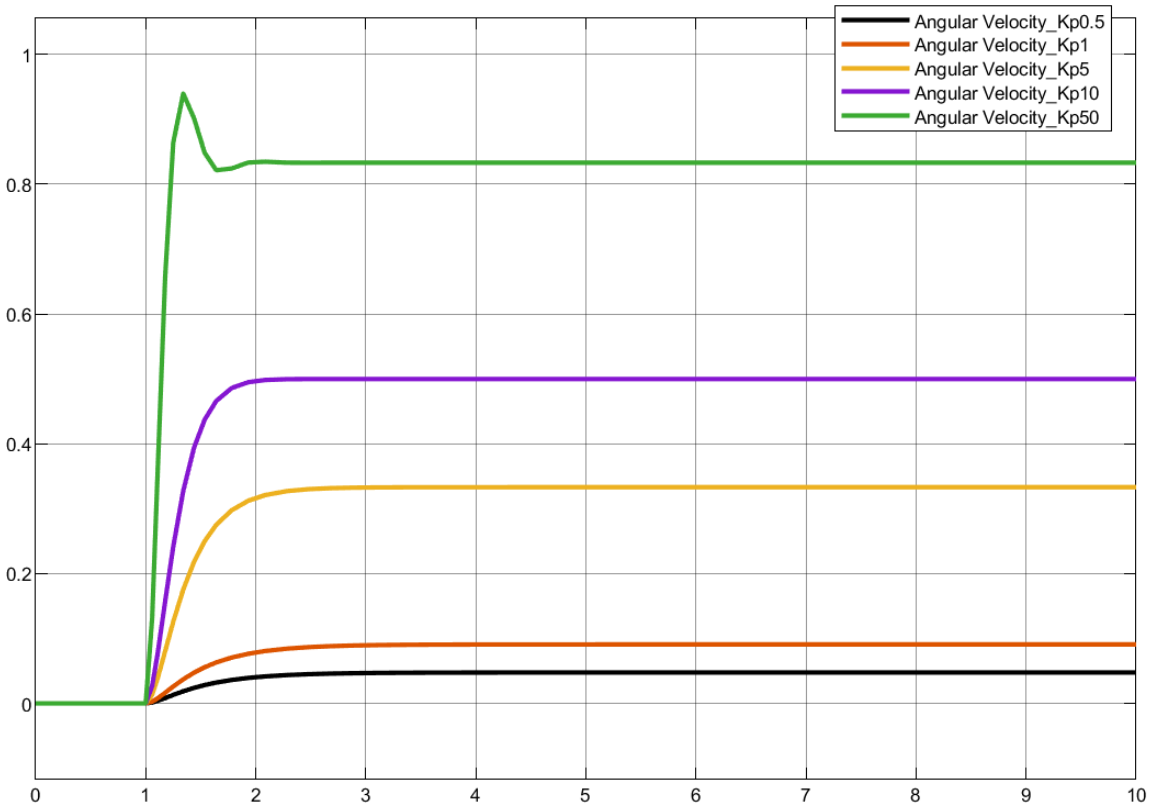


Figure 8 Step Responses for P Controller — $K_p = 0.5, 1, 5, 10, 50$

5.3.5 Performance Specifications

Table 4 P Controller Performance Specifications for Varying K_p

K_p	Rise Time (s)	Settling Time (s)	Overshoot (%)	Steady-State Speed
0.5	1.08	2.96	0	0.0476
1	1.03	2.85	0	0.0908
5	0.71	2.26	0	0.3331
10	0.50	1.84	0.006	0.4998
50	0.17	1.53	12.79	0.9398

5.3.6 Observations:

- P control reduces SSE but never eliminates it. At $K_p=50$, the steady-state speed reached 0.9398 rad/s, much closer to 1.0 but still 6% short. No matter how high K_p is pushed, the P controller cannot achieve zero steady-state error for a step input. This is a mathematical property of proportional control, not a tuning problem.
- Increasing K_p makes the system faster but eventually introduces overshoot. At $K_p=10$, overshoot was negligible (0.006%). At $K_p=50$, it jumped to 12.79%. The system transitioned from overdamped to underdamped behavior as K_p crossed a critical threshold — the same breakaway point phenomenon observed in Project 02's pole analysis.
- $K_p=0.5$ produced a larger SSE than unity feedback.
- The SSE at $K_p=0.5$ was $1 - 0.0476 = \mathbf{0.9524}$, compared to $1 - 0.0908 = \mathbf{0.9092}$ for unity feedback. Lower K_p means less aggressive correction, so the motor reaches a lower final speed.

5.4 PI Controller Analysis

The Simulink PID block was used with K_d set to zero. K_p was fixed at 10, and K_i was varied from 1 to 50. The integral term accumulates the error over time and adds a correction proportional to the accumulated error. This means that as long as any error remains, the integral term keeps growing and applying additional input — until the error reaches exactly zero.

5.4.1 PI Controller Simulink Block Diagram

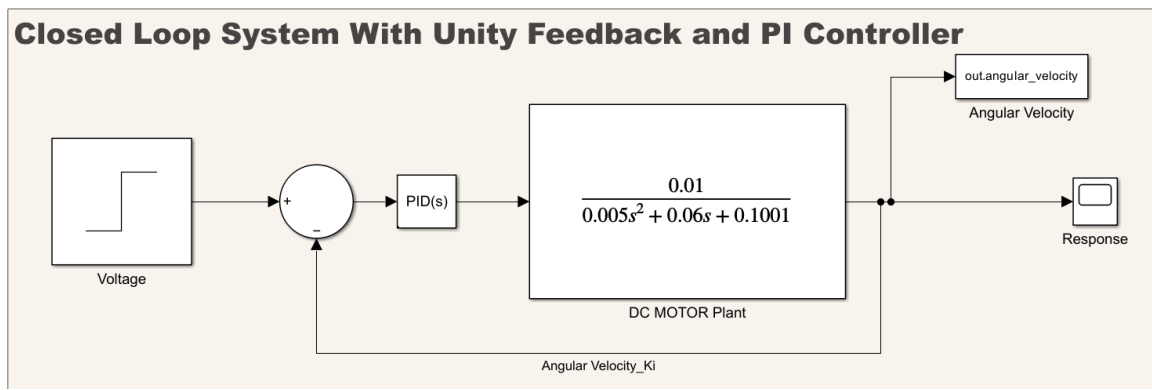


Figure 9 Simulink Block Diagram — PI Controller

5.4.2 PI Controller Simulink Block Diagram for varying K_i

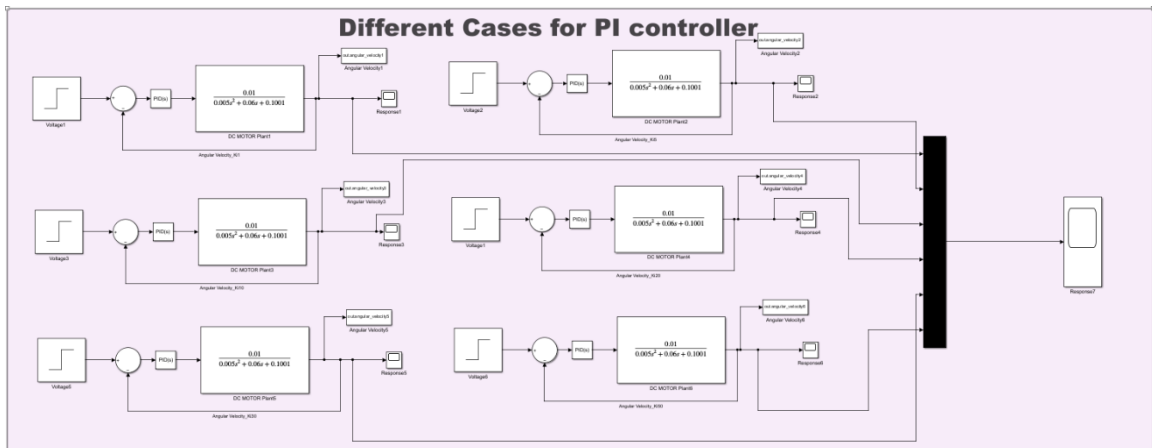


Figure 10 Simulink Block Diagram — PI Controller — $K_i = 1, 5, 10, 20, 30, 50$ ($K_p=10$)

5.4.3 PI Controller Step Responses for All Ki Values

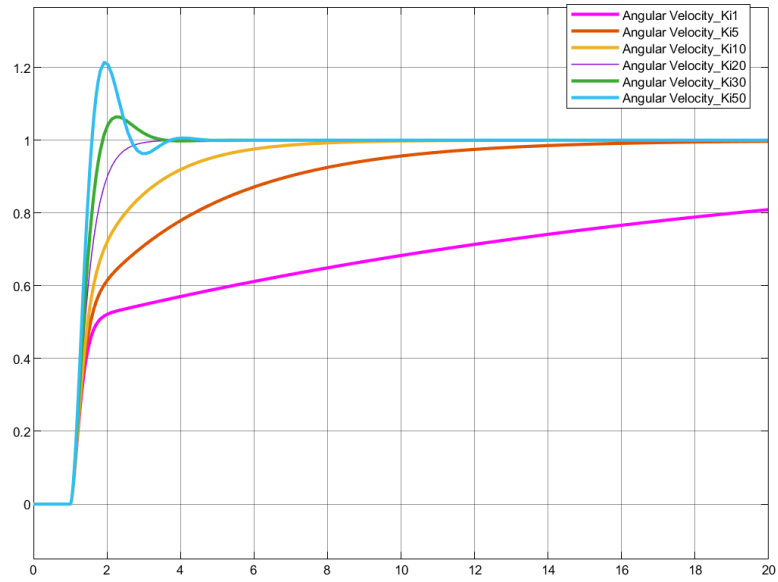


Figure 11 Step Responses for PI Controller — $K_i = 1, 5, 10, 20, 30, 50$ ($K_p=10$)

5.4.4 Performance Specifications

Table 5 PI Controller Performance Specifications — Varying K_i with $K_p=10$

K_i ($K_p=10$)	Rise Time (s)	Settling Time (s)	Overshoot (%)	SS = 1.0?
1	11.92	18.40	0	No — 0.809
5	5.70	12.37	0	Almost — 0.997
10	2.52	6.32	0	Yes — 1.000
20	0.89	2.60	~0	Yes — 1.000
30	0.58	2.99	6.40	Yes but overshoots
50	0.41	3.37	21.41	Yes but badly overshoots

5.4.5 Observations

- Integral action eliminates steady-state error completely. At $K_i=10$, the peak value reached exactly 1.000 rad/s with zero overshoot. This is the defining characteristic of integral control: **it keeps correcting until the output equals the reference, regardless of plant parameters.**
- The sweet spot for this plant is $K_i=20$. At $K_i=20$, the system achieves zero SSE, a rise time of 0.89 s, and negligible overshoot (effectively zero). Beyond $K_i=20$, integrator windup begins to dominate.
- High K_i causes integrator windup and overshoot. At $K_i=30$, overshoot jumped to 6.4%; at $K_i=50$, it reached 21.4%. This happens because the integrator accumulates a large correction during the initial rise and cannot immediately cancel that accumulated value when the output crosses the reference, causing the system to overshoot significantly before settling back.

5.5 PD Controller Analysis

The PID block was used with K_i set to zero. K_p was fixed at 10, and K_d was varied from 0.01 to 10. The derivative term reacts to the rate of change of error — it is essentially a predictive brake that applies resistance proportional to how fast the output is approaching the reference.

5.5.1 PD Controller Simulink Block Diagram

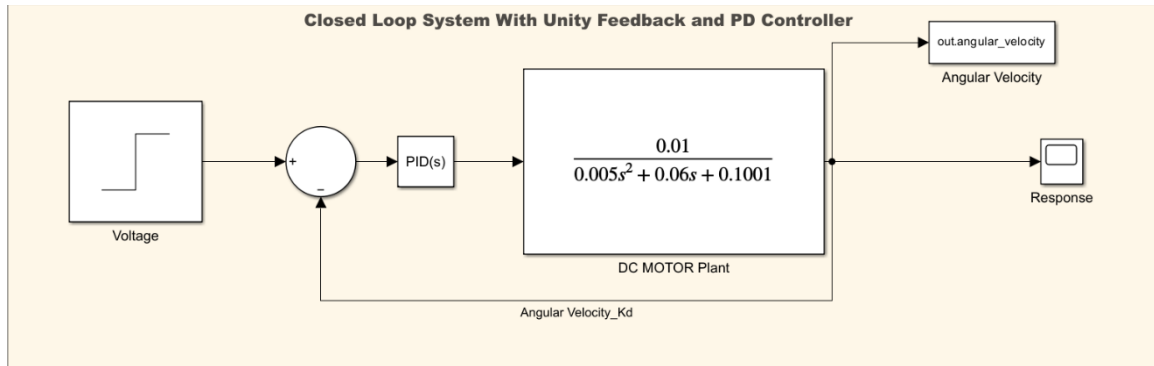


Figure 12 Simulink Block Diagram — PD Controller

5.5.2 PD Controller Simulink Block Diagram for varying K_d

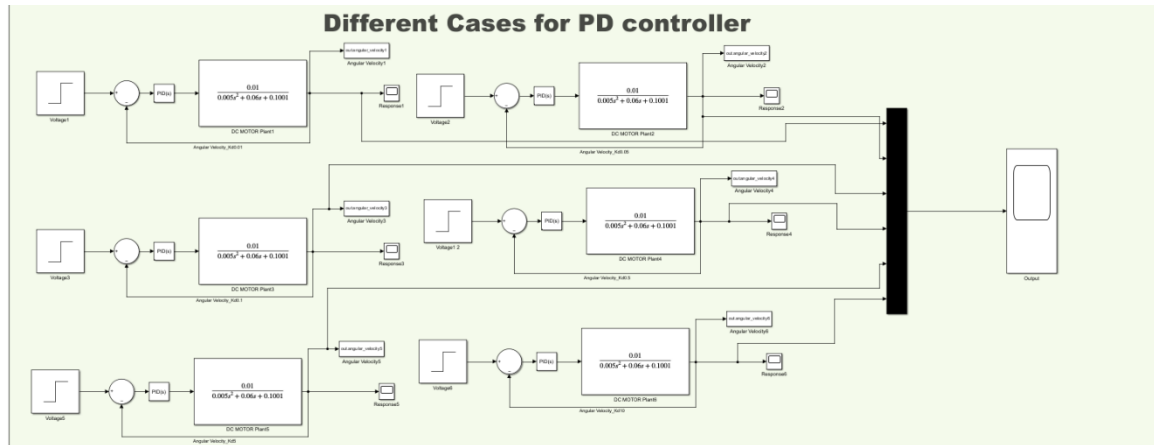


Figure 13 Simulink Block Diagram — PD Controller — $K_d = 0.01, 0.05, 0.1, 0.5, 5, 10$ ($K_p=10$)

5.5.3 PD Controller Step Responses for All Kd Values

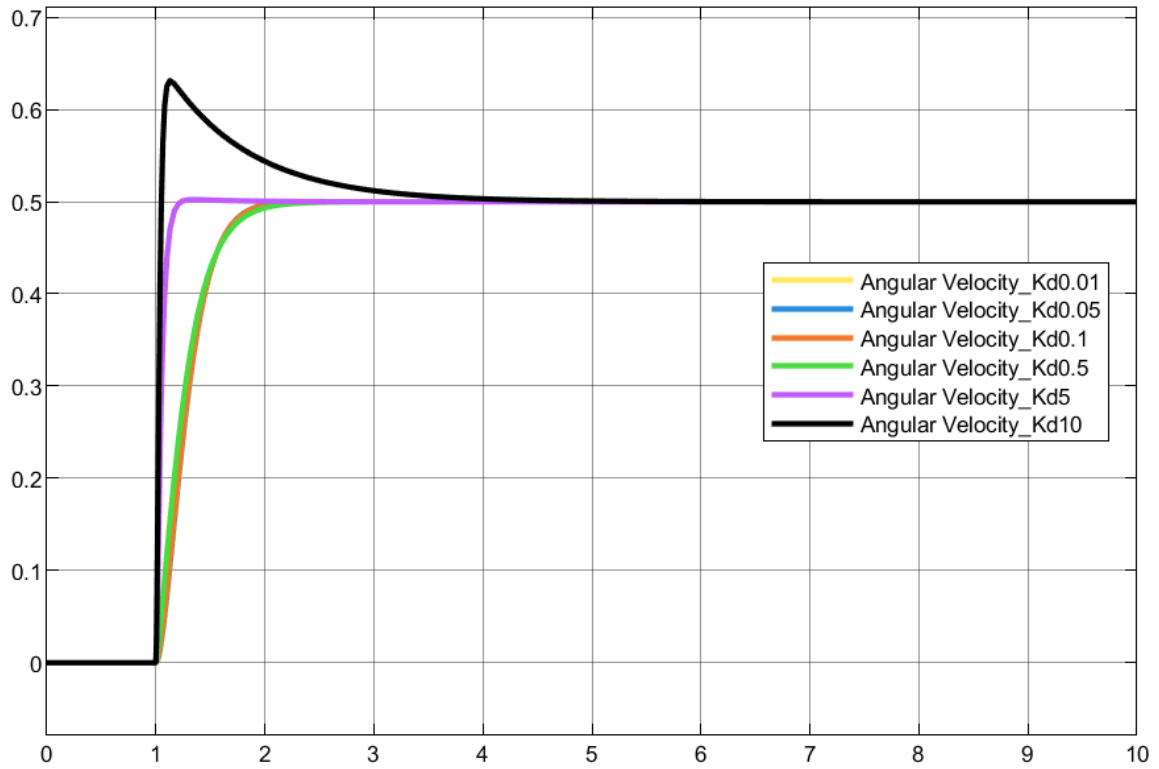


Figure 14 Step Responses for PD Controller — $K_d = 0.01, 0.05, 0.1, 0.5, 5, 10$ ($K_p=10$)

5.5.4 Performance Specifications

Table 6 PD Controller Performance Specifications — Varying K_d with $K_p=10$

K_d ($K_p=10$)	Rise Time (s)	Settling Time (s)	Overshoot (%)
0.01	0.493	1.831	0.007
0.05	0.497	1.838	0.003
0.10	0.502	1.848	0.001
0.50	0.529	1.915	~0
5.00	0.099	1.168	0.479
10.00	0.036	3.145	26.31

5.5.5 Observations:

- Small K_d reduces overshoot smoothly without significantly affecting rise time or settling time. In the range $K_d=0.01$ to $K_d=0.5$, the system behavior is nearly identical to P-only control but with virtually eliminated overshoot. The derivative term is acting as a gentle brake, smoothing the approach to the reference.
- Large K_d causes the system to become hypersensitive and eventually unstable in behavior. At $K_d=10$, the rise time dropped to 0.036 s, but settling time jumped to 3.145 s and overshoot reached 26.3%. This is the derivative over-braking effect: *the controller sees a very fast error change at $t=1s$ (when the step is applied) and applies massive braking, initially slowing the response dramatically. The system then lurches forward and oscillates around the reference.*
- D control does not fix steady-state error. The steady-state speed remained at approximately 0.4998 rad/s for all K_d values, confirming that derivative action has no influence on steady-state performance. D control is a transient tool only.

The optimal K_d for this plant is between 0.1 and 0.5, where overshoot is effectively eliminated without sacrificing response speed.

5.6 Full PID Controller Tuning

The three control actions were combined using the knowledge gained from the individual studies. The starting point was **Kp=10, Ki=20, Kd=0.1**, representing the sweet spots identified in the P, PI, and PD analyses. Three nearby gain combinations were tested to find the final tuning.

5.6.1 PID Controller Simulink Block Diagram

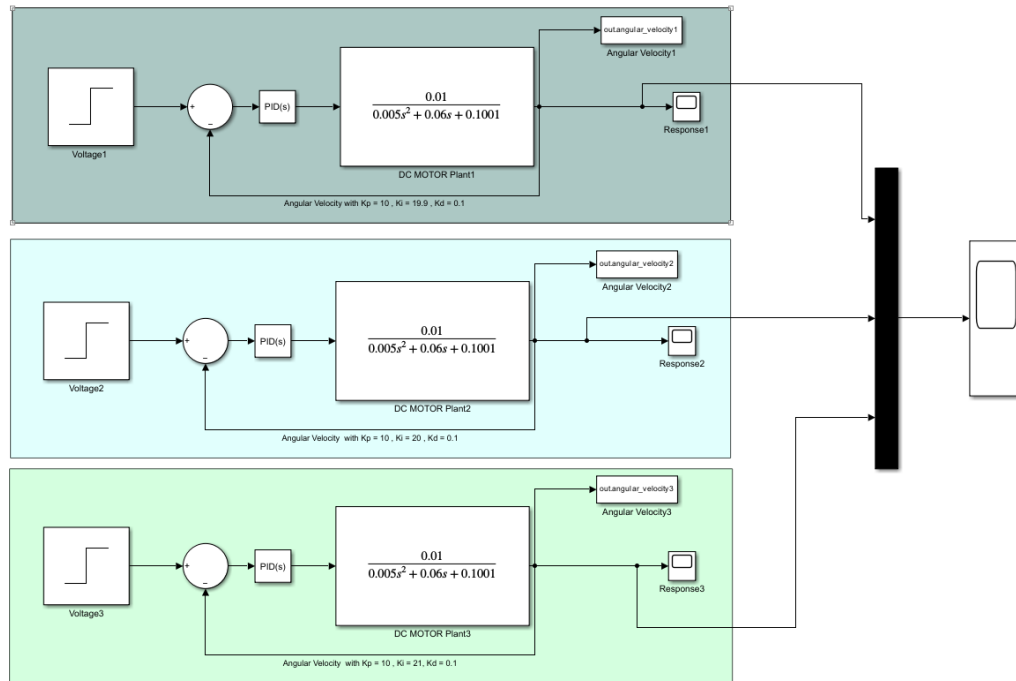


Figure 15 Simulink Block Diagram — Full PID Controller

5.6.2 PID Step Responses — Three Gain Combinations

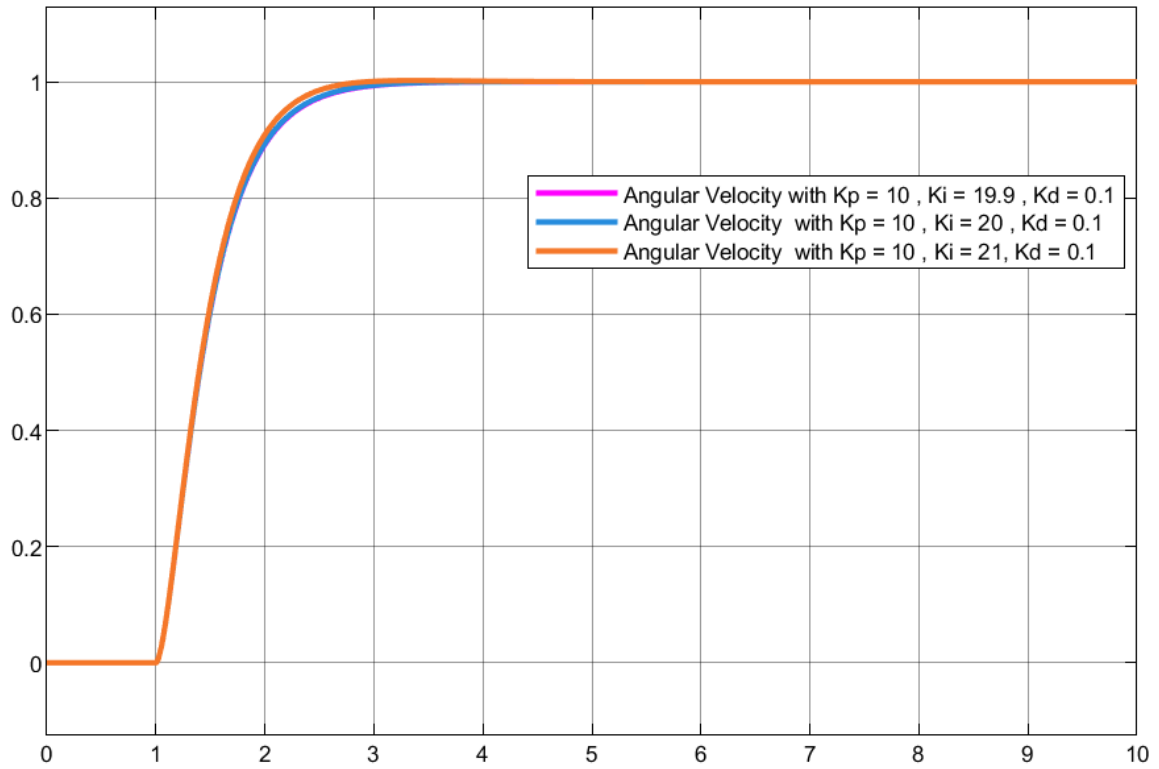


Figure 16 Step Responses for Three PID Gain Combinations

5.6.3 Performance Specifications

5.6.3.1 Comparison of different combinations

Table 7 PID Tuning Comparison — Three Gain Combinations

K_p	K_i	K_d	Rise Time (s)	Settling Time (s)	Overshoot (%)	SS Error
10	19.9	0.1	0.916	2.616	0.004	0%
10	20.0	0.1	0.910	2.596	0.009	0%
10	21.0	0.1	0.857	2.425	0.212	0%

5.6.3.2 Selected Gains

The selected final tuning is **Kp=10, Ki=20, Kd=0.1**. All four performance targets were met simultaneously:

Table 8 Final PID Tuning — Performance Target Verification

Metric	Result	Target	Status
Rise Time	0.91 s	< 1 s	PASS
Settling Time	2.60 s	< 3 s	PASS
Overshoot	0.009%	< 5%	PASS
Steady-State Error	0%	0%	PASS

5.7 Complete Controller Comparison

The following table summarizes the performance of all controller configurations tested in this project, providing a direct comparison that reveals the contribution of each control action.

Table 9 Complete Controller Comparison — All Configurations

Controller	Rise Time	Settling Time	Overshoot	SS Error
Open Loop	1.14 s	3.07 s	0%	90.02%
Closed Loop — Unity	1.03 s	2.85 s	0%	90.92%
P only (Kp=10)	0.50 s	1.84 s	0.006%	50.02%
PI (Kp=10, Ki=20)	0.89 s	2.60 s	~0%	0%
PD (Kp=10, Kd=0.1)	0.50 s	1.85 s	0.001%	50.02%
PID (Kp=10, Ki=20, Kd=0.1)	0.91 s	2.60 s	0.009%	0%

5.7.1 Observations

- The table reveals the individual contribution of each control action clearly. P control reduced steady-state error from 90% to 50% but could not eliminate it. PI control eliminated steady-state error entirely but PD control could not. The full PID controller achieved all targets simultaneously by combining the strengths of all three actions.
- PID is not always better than PI. The derivative term adds value when the plant dynamics are fast, complex, or prone to oscillation. For slow overdamped plants like this DC motor, PI alone is often sufficient.

5.8 pidtune() Automatic Tuning Comparison

MATLAB's `pidtune()` function was used to automatically compute PID gains for the same DC motor plant.

The function returned **Kp=19.5, Ki=53, Kd=1.57**.

5.8.1 Performance Specifications

Table 10 Manual PID Tuning vs pidtune() Automatic Tuning

	Kp	Ki	Kd	Rise Time	Settling Time	Overshoot	SS Error
Manual Tuning	10	20	0.1	0.91 s	2.60 s	0.009%	0%
pidtune() Auto	19.5	53	1.57	0.46 s	1.60 s	6.39%	0%

5.8.2 Gain Comparison Analysis:

1. **Kp (19.5 vs 10):** pidtune() uses higher proportional gain to achieve faster response. This explains the shorter rise time of 0.46 s versus 0.91 s.
2. **Ki (53 vs 20):** The higher integral gain is sustainable at Kp=19.5 because the stronger proportional action drives the system faster, reducing the time the integrator spends accumulating error. At Kp=10, Ki=53 would cause severe integrator windup.
3. **Kd (1.57 vs 0.1):** Higher derivative gain is necessary to counteract the increased oscillation tendency from the more aggressive Kp and Ki values. Without Kd=1.57, the pidtune() gains would produce much larger overshoot. The three gains are not independent — they form a balanced set.
4. Neither tuning is universally superior. The automatic tuning is twice as fast but carries 6.39% overshoot. For an aircraft control surface actuator, 6.39% overshoot means the surface moves past its commanded position before settling — an unacceptable behavior in flight. *The manual tuning sacrifices speed for smoothness, which is the correct priority in aerospace applications.*

5.9 Disturbance Rejection

The final and most practically important test was disturbance rejection. A step disturbance was injected at the plant input at $t=5s$, after the motor had already settled at 1.0 rad/s. This simulates a sudden mechanical load — equivalent to an aircraft actuator suddenly encountering increased aerodynamic resistance, or a reaction wheel motor experiencing a gravitational disturbance torque.

5.9.1 Disturbance Rejection Simulink Block Diagram

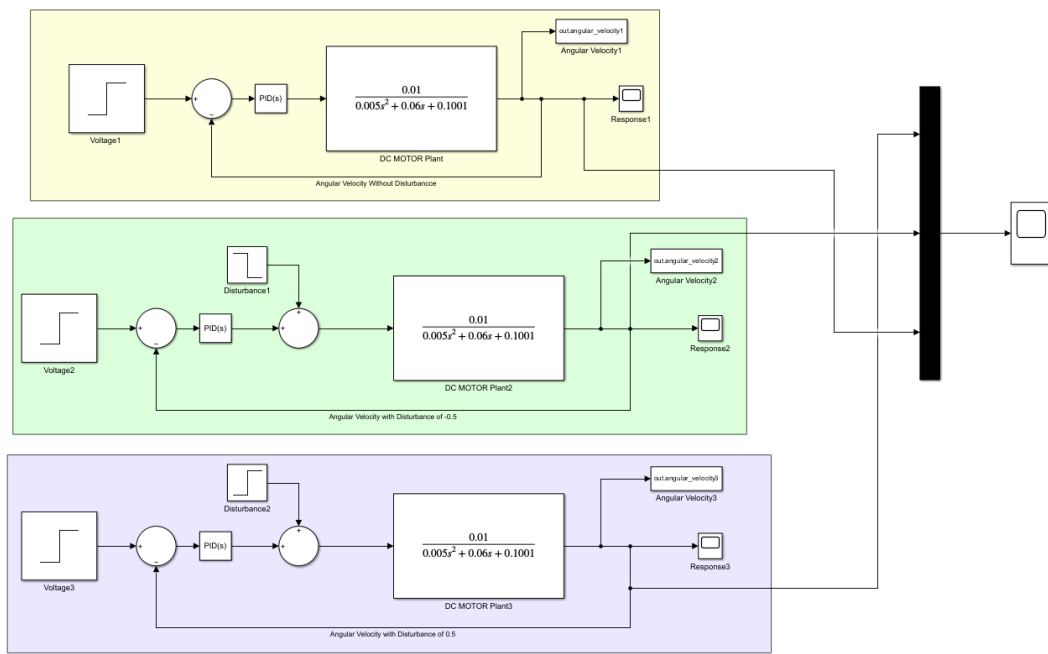


Figure 17 Simulink Block Diagram — Disturbance Rejection Test

5.9.2 Disturbance Rejection Response

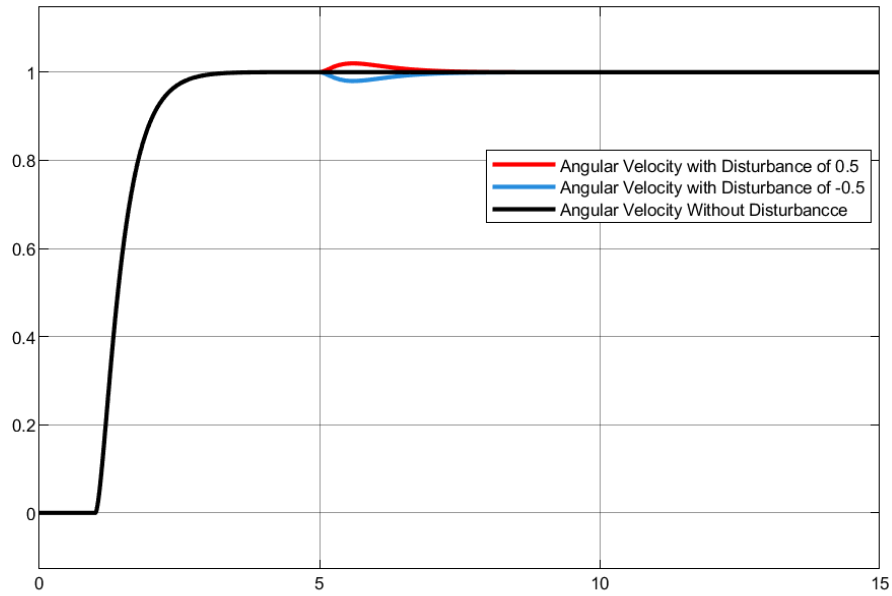


Figure 18 Motor Speed Response with Disturbance

5.9.3 Disturbance Rejection Results

Table 11 Disturbance Rejection Results — Final Speed After Recovery

Condition	Speed at t=5s (before disturbance)	Speed immediately after disturbance	Final Speed	Recovery
No Disturbance	1.000 rad/s	—	1.000 rad/s	N/A
Disturbance = -0.5	1.000 rad/s	~ 0.950 rad/s	1.000 rad/s	Complete
Disturbance = $+0.5$	1.000 rad/s	~ 1.020 rad/s	1.000 rad/s	Complete

5.9.4 Observations:

- Both positive and negative disturbances were completely rejected. In both cases, the motor speed returned to exactly 1.0 rad/s after the disturbance — zero steady-state error was maintained even under external perturbation.
- The negative disturbance caused a brief speed drop to approximately 0.95 rad/s. The PID controller detected this as an error (reference=1.0, output=0.95, error=+0.05), increased the control input, and drove the motor back to 1.0 rad/s within approximately 1 second.
- The positive disturbance caused a brief speed increase to approximately 1.02 rad/s. The controller detected a negative error, reduced the control input, and the motor settled back to 1.0 rad/s.
- This result demonstrates the most powerful property of integral control: it does not just reject the initial steady-state error at startup — it continuously monitors and corrects for any error that appears at any time during operation, including errors caused by external disturbances. An open-loop system, or a P-only controller, would permanently settle at a different speed after the disturbance. The PID controller returns to exactly 1.0 rad/s every

6 Aerospace Relation

The PID controller designed in this project is not an academic exercise — it represents the exact class of controller used in real aerospace systems. Understanding its properties, limitations, and tuning trade-offs is directly applicable to the following aerospace engineering contexts.

Aircraft Flight Control Actuators: Modern commercial and military aircraft use electrically actuated control surfaces. The elevator, aileron, and rudder are moved by servo motors that must reach commanded positions quickly, without overshoot, and maintain those positions despite aerodynamic disturbance forces. The disturbance rejection test in this project directly models this scenario. The choice to prioritize manual tuning over `pidtune()` — accepting slower response in exchange for zero overshoot — is the exact trade-off made by flight control engineers. An elevator that overshoots by 6% before settling could briefly pitch the aircraft in the wrong direction.

UAV Propulsion Control: Multirotor UAVs use PID controllers on each motor to maintain desired thrust and attitude. The motors must respond rapidly to commanded speed changes (fast rise time) while rejecting disturbances caused by wind gusts (disturbance rejection). The integrator windup problem observed at high K_i values in this project is a real concern in UAV motor controllers — windup during rapid maneuvers can cause temporary loss of attitude control.

Satellite Reaction Wheels: Reaction wheel assemblies use DC motors to spin wheels at precisely controlled speeds, generating angular momentum for attitude control without consuming propellant. The motor must maintain exact speed despite bearing friction disturbances and changing gravitational gradients. The integral term in the PID controller ensures that friction-induced speed drops are automatically corrected — exactly the behavior demonstrated in the disturbance rejection test.

Rocket Thrust Vector Control: Gimbal actuators in rocket engines must precisely position the nozzle to generate the correct steering torques. These actuators are driven by electric motors controlled by PID loops. The requirement for zero steady-state error (so the

nozzle holds exactly the commanded angle) and fast response (so the rocket responds quickly to trajectory corrections) mirrors exactly the design targets pursued in this project.

7 Conclusions

This project successfully completed the design, implementation, and analysis of a PID speed controller for a DC motor using MATLAB and Simulink. The following engineering conclusions were established:

- Open-loop control is insufficient for precision speed regulation. The motor reached only 0.0998 rad/s for a 1 V command, representing a 90% steady-state error. No amount of manual input adjustment can reliably correct for this because the error depends on motor parameters, not on the desired speed.
- Unity feedback alone does not fix steady-state error. Closing the loop without a controller only redistributes the error and is mathematically equivalent to a P controller with $K_p=1$.
- P control reduces steady-state error but cannot eliminate it. As K_p increases, the response becomes faster and the error decreases, but a residual error always remains. At high K_p , overshoot appears as the system transitions from overdamped to underdamped behavior.
- Integral action is the only mechanism that can drive steady-state error to exactly zero. The integral term accumulates error over time and continues to apply corrections until the output matches the reference exactly. However, excessively high K_i causes integrator windup, producing unacceptable overshoot.
- Derivative action reduces overshoot by braking the approach to the reference, but has no effect on steady-state error. Excessive K_d causes over-braking and paradoxically increases settling time and overshoot.
- The final manual PID tuning ($K_p=10$, $K_i=20$, $K_d=0.1$) met all four performance targets simultaneously: rise time 0.91 s, settling time 2.60 s, overshoot 0.009%, steady-state error 0%.
- Automatic tuning (pidtune) optimizes for speed but sacrifices smoothness. The resulting 6.39% overshoot makes the automatic tuning unsuitable for aerospace actuator applications where overshoot can have safety implications.

- The tuned PID controller completely rejects disturbances. Both positive and negative step disturbances of magnitude 0.5 applied at $t=5s$ were fully rejected, with the motor returning to exactly 1.0 rad/s in both cases.

8 Future Work

Project 03 established a complete working knowledge of PID feedback control using a DC motor as the plant. The skills developed here — predicting controller behavior, tuning gains systematically, interpreting pole movement, and testing disturbance rejection — form the foundation for a structured fifteen-project roadmap leading from classical control fundamentals to complete aerospace flight control systems.

Project 04 — Aircraft Pitch Control

The PID design methodology will be applied to a linearized aircraft longitudinal dynamics model. The pitch dynamics plant is higher order than the DC motor and may contain open-loop poles that are marginally stable or lightly damped, requiring more careful gain selection. This project directly connects the motor control skills to aerospace plant dynamics for the first time.

Project 05 — Root Locus Design

Formal root locus analysis will replace the trial-and-error gain tuning used in this project. By plotting how closed-loop poles move as a function of controller gain, it becomes possible to select gains analytically — predicting exactly what rise time and damping ratio will result from a given K_p before running a single simulation.

Project 06 — Bode and Nyquist Analysis

Gain margin and phase margin will be computed to formally characterize how much parameter variation or time delay the closed-loop system can tolerate before becoming unstable. These stability margins are a mandatory design requirement in aerospace control systems and cannot be assessed from step response alone.

Project 07 — State Space Modeling

The DC motor and aircraft plants will be re-modeled in state space form using the A, B, C, D matrix representation. State space is the language of modern control theory and is

required for all subsequent advanced controller designs. It also enables simulation of multi-input multi-output systems, which are essential for full aircraft and rocket control.

Project 08 — Pole Placement

Full state feedback will be used to place closed-loop poles at precisely specified locations in the complex plane, achieving exact desired transient performance. Unlike PID tuning, pole placement provides a direct mathematical relationship between desired performance specifications and required controller gains.

Project 09 — LQR Optimal Control

The Linear Quadratic Regulator will be designed by minimizing a cost function that balances control effort against state error. LQR produces optimal state feedback gains automatically and is one of the most widely used controllers in aerospace applications including aircraft autopilots, satellite attitude control, and rocket landing systems.

Project 10 — Kalman Filter

A Kalman filter will be designed to estimate system states from noisy sensor measurements. In real hardware, IMU sensors on rockets and UAVs produce noisy angular rate and acceleration data. The Kalman filter fuses this noisy data with a system model to produce clean state estimates — it is the essential bridge between simulation and physical hardware implementation, and is directly required for the Teknofest VLR rocket.

Project 11 — UAV Attitude Control

A full quadrotor attitude control system will be built in Simulink, including separate PID controllers for roll, pitch, and yaw axes, motor mixing logic, and nonlinear rigid body dynamics. This project introduces multi-axis control and the coupling between axes that does not exist in single-input single-output systems.

Project 12 — Rocket Attitude Control

A vertical landing rocket attitude control system — directly modeled on the Teknofest VLR architecture — will be simulated in Simulink. This includes cold gas RCS thruster modeling with PWPF modulation, TVC gimbal dynamics, cascaded PID loops for attitude

and position, and 6DOF rigid body dynamics. This project is the direct simulation validation tool for the actual competition hardware.

Project 13 — Satellite Attitude Control

Reaction wheel based attitude control using quaternion kinematics will be implemented. Satellite control introduces zero-gravity dynamics, conservation of angular momentum constraints, and the requirement for propellant-free maneuvering — all of which require modern control techniques beyond classical PID.

Project 14 — Missile Guidance

Proportional navigation guidance law will be implemented and analyzed. Missile guidance connects control theory to trajectory optimization and introduces the concept of commanded acceleration as a control input, which differs fundamentally from the position and speed control studied in earlier projects.

Project 15 — Complete Flight Control System

A complete integrated flight control system will be assembled combining autopilot, attitude control, guidance, and navigation into a single Simulink environment. This capstone project represents the synthesis of all fifteen projects and produces a portfolio piece demonstrating end-to-end aerospace control system design capability.

9 Appendix

9.1 MATLAB Code

```
%% ----- Closed Loop System Performance Anlysis of DC Motor
System----
clc

%% ----- Closed Loop DC Motor System with only P controller----
disp("System Performance when Kp = 1 is :")
info_kp =
stepinfo(out.angular_velocity.Data,out.angular_velocity.Time);
disp(info_kp)

kp_value = [0.5,1,5,10,50];
for i = 1:length(kp_value)
    signal = out.(sprintf('angular_velocity%d',i));
    info = stepinfo(signal.Data, signal.Time);
    fprintf("System Performance when Kp = %d is :\n",kp_value(i))
    disp(info)
end

%% ----- Closed Loop DC Motor System with only PI controller----

ki_value = [1,5,10,20,30,50];
for i = 1:length(ki_value)
    signal = out.(sprintf('angular_velocity%d',i));
    info = stepinfo(signal.Data, signal.Time);
    fprintf("System Performance when Kp = 10 & Ki = %d is
:\n",ki_value(i))
    disp(info)
end

%% ----- Closed Loop DC Motor System with only PD controller----

kd_value = [0.01,0.05,0.1,0.5,5,10];
for i = 1:length(kd_value)
    signal = out.(sprintf('angular_velocity%d',i));
    info = stepinfo(signal.Data, signal.Time);
    fprintf("System Performance when Kp = 10 & Kd = %d is
:\n",kd_value(i))
    disp(info)
end

%% ----- Closed Loop DC Motor System with PID controller----
```

```

kp1 = 10 ; ki1 = 19.9 ; kd1 = 0.1 ;
info1 = stepinfo(out.angular_velocity1.Data,
out.angular_velocity1.Time);
fprintf("System Performance when Kp = 10 , Ki = 19.9 , Kd = 0.1
is :\n")
disp(info1)

```

```

kp2 = 10 ; ki2 = 20 ; kd2 = 0.1 ;
info2 = stepinfo(out.angular_velocity2.Data,
out.angular_velocity2.Time);
fprintf("System Performance when Kp = 10 , Ki = 20 , Kd = 0.1 is
:\n")
disp(info2)

```

```

kp3 = 10 ; ki3 = 21 ; kd3 = 0.1 ;
info3 = stepinfo(out.angular_velocity3.Data,
out.angular_velocity3.Time);
fprintf("System Performance when Kp = 10 , Ki = 21 , Kd = 0.1 is
:\n")
disp(info3)

```

%% PID Tune

```

J=0.01; b=0.1; R=1; L=0.5; Kt=0.01; Ke=0.01;
G = tf(Kt, [(J*L) (J*R+b*L) (b*R+Kt*Ke)]);
C = pidtune(G, 'PID');
T = feedback(C*G,1);
stepinfo(T)

```

%% ---- Disturbance Rejection Case ---

```

kp1 = 10 ; ki1 = 20 ; kd1 = 0.1 ;
info1 = stepinfo(out.angular_velocity1.Data,
out.angular_velocity1.Time);
fprintf("System Performance without the Disturbance is :\n")
disp(info1)

```

```

% Step Disturbance of value -0.5 at t = 5s
info2 = stepinfo(out.angular_velocity2.Data,
out.angular_velocity2.Time);

```

```
fprintf("System Performance when Disturbance of -0.5 is  
involved:\n")  
disp(info2)  
  
% Step Disturbance of value 0.5 at t = 5s  
info3 = stepinfo(out.angular_velocity3.Data,  
out.angular_velocity3.Time);  
fprintf("System Performance when Disturbance of 0.5 is  
involved:\n")  
disp(info3)
```